

Delta Lake Incremental Processing using Databricks

Objective

Implement an end-to-end incremental data processing pipeline using Apache Spark and Delta Lake in Databricks. The project demonstrates loading a CSV dataset into a Delta table, performing data cleaning, generating incremental data, applying the MERGE operation, and validating the final dataset.

Dataset

The project uses a shopping products dataset (Combined_dataset.csv). It contains product information such as Product ID, Title, Description, Rating, Price, Seller Details, Category, and related metadata. Incremental records are generated programmatically to simulate updates and inserts.

Steps Performed

1. Loaded the CSV dataset into a Spark DataFrame.

```
Read csv file

04:57 PM (24s) 3

# Dataset_path
dataset_path = "/Volumes/workspace/default/shopping_dataset/Combined_dataset.csv"

# Read CSV
df = spark.read.csv(
    dataset_path,
    header=True,
    inferSchema=True
)

display(df)

> See performance (2) ✓ Checked for improvements
df: pyspark.sql.connect.dataframe.DataFrame = [product_id: integer, title: string ... 22 more fields]
```

	product_id	title	product_description	rating
1	8376765	Lino Perros	Women Navy Blue Solid Backpack	3.8
2	9136281	Tommy Hilfiger	Unisex Navy Blue Striped Backpack	4.5
3	17633752	Lavie	Aries Women Pink Mini Backpack	4.4
4	1376949	F Gear	Unisex Navy & Grey Printed Burner Backpack	4.4
5	13939916	MYTRIDENT	Men Blue Solid Bath Robe	4.7
6	17198778	H&M	Beige & Tan Solid Stoneware Soap Dispenser	4.5
7	19851824	AVI Living	Grey Solid 500 GSM Pure Cotton Bath Towel	4
8	18602872	My Room	Yellow & Green Floral 140 TC King Bedsheet with 2 Pillow Covers	4.7
9	18602850	My Room	Ethnic Print 144 TC Super King Bedsheet with 2 Pillow Covers	4.5
10	8961147	Story@home	Pink & White Floral 160 TC Cotton 1 King Bedsheet with 2 Pillow Covers	3.7
11	8430275	MYTRIDENT	Sea Green Floral Flat 120 TC Cotton 1 Queen Bedsheet with 2 Pillow Covers	4.4
12	18346118	Arrabi	Pink & White Graphic 300 TC King Bedsheet with 2 Pillow Covers	0
13	18327434	BELLA CASA	Unisex Blue Bedsheets	0
14	18492504	ROMEE	Blue & Beige Conversational 160 TC King Bedsheet with 2 Pillow Covers	3.9

1,000 rows | 24.456s runtime Refreshed 2 hours ago

```
04:57 PM (1s) 4

print("Number of rows :", df.count())
print("Number of columns :", len(df.columns))

> See performance (1) ✓ Checked for improvements

Number of rows : 1000
Number of columns : 24
```

```
df.printSchema()

root
 |-- product_id: integer (nullable = true)
 |-- title: string (nullable = true)
 |-- product_description: string (nullable = true)
 |-- rating: double (nullable = true)
 |-- ratings_count: integer (nullable = true)
 |-- initial_price: integer (nullable = true)
 |-- discount: integer (nullable = true)
 |-- final_price: string (nullable = true)
 |-- currency: string (nullable = true)
 |-- images: string (nullable = true)
 |-- delivery_options: string (nullable = true)
 |-- product_details: string (nullable = true)
 |-- breadcrumbs: string (nullable = true)
 |-- product_specifications: string (nullable = true)
 |-- amount_of_stars: string (nullable = true)
 |-- what_customers_said: string (nullable = true)
 |-- seller_name: string (nullable = true)
 |-- sizes: string (nullable = true)
 |-- videos: string (nullable = true)
 |-- seller_information: string (nullable = true)
```

display(df.describe())

See performance (1) ✓ Checked for improvements

	summary	product_id	title	product_description	rating	ratings_count
1	count	1000	1000	1000	1000	1000
2	mean	1.7134372582E7	null	null	3.6215	76.691
3	stddev	3813765.757472536	null	null	1.37440049720841...	241.1142
4	min	586846	4WRD by Dressberry	100% Natural & Pure Amla Powder for Hair & Skin - 100 g	0.0	0
5	max	23713666	100% Natural & Pure Amla Powder for Hair & Skin - 100 g	100% Natural & Pure Amla Powder for Hair & Skin - 100 g	5.0	1115

5 rows | 2.684s runtime Refreshed 2 hours ago

2. Stored the data as a Delta Table.

```
delta_path = "/Volumes/workspace/default/shopping_dataset/product_master_delta"

df.write.format("delta") \
  .mode("overwrite") \
  .save(delta_path)
```

See performance (1) ✓ Checked for improvements

```
delta_df = spark.read.format("delta").load(delta_path)

display(delta_df)
```

delta_df: pyspark.sql.connect.dataframe.DataFrame = [product_id: integer, title: string ... 22 more fields]

	product_id	title	product_description	rating	ratings_count
1	8376765	Lino Perros	Women Navy Blue Solid Backpack	3.8	1
2	9136281	Tommy Hilfiger	Unisex Navy Blue Striped Backpack	4.5	1
3	17633752	Lavie	Aries Women Pink Mini Backpack	4.4	1
4	1376949	F Gear	Unisex Navy & Grey Printed Burner Backpack	4.4	1
5	13939916	MYTRIDENT	Men Blue Solid Bath Robe	4.7	1
6	17198778	H&M	Beige & Tan Solid Stoneware Soap Dispenser	4.5	1
7	19851824	AVI Living	Grey Solid 500 GSM Pure Cotton Bath Towel	4	1
8	18672872	Mu Broom	Yellow & Green Floral 140 TC King Bedsheet with 2 Pillow Covers	4.7	1

3. Performed basic data cleaning (handled null values and removed duplicates).

Performing basic cleaning

Check null counts

```
from pyspark.sql.functions import col, count, when

null_counts = delta_df.select([
    count.when(col(c).isNull(), c).alias(c)
    for c in delta_df.columns
])

display(null_counts)
```

> [See performance \(1\)](#) ✓ Checked for improvements

> null_counts: pyspark.sql.connect.dataframe.DataFrame = [product_id: long, title: long ... 22 more fields]

	product_id	title	product_description	rating	ratings_count	initial_price	discount
1	0	0	0	0	0	0	121

1 row | 1.737s runtime Refreshed 2 hours ago

Fill null values

```
delta_df = delta_df.fillna({
    "rating": 0.0,
    "ratings_count": 0,
    "seller_name": "Unknown",
    "category": "Unknown"
})
```

> delta_df: pyspark.sql.connect.dataframe.DataFrame = [product_id: integer, title: string ... 22 more fields]

Drop duplicate values

```
clean_df = delta_df.dropDuplicates(["product_id"])
```

> clean_df: pyspark.sql.connect.dataframe.DataFrame = [product_id: integer, title: string ... 22 more fields]

Saving the cleaned data back to delta table

```
clean_df.write \
    .format("delta") \
    .mode("overwrite") \
    .option("overwriteSchema", "true") \
    .save(delta_path)
```

> [See performance \(1\)](#) ✓ Checked for improvements

4. Created an incremental dataset with updated and new records.

⋮ A second dataset is created to simulate new incoming data. It contains:

- Updated records for existing products.
- New products that do not exist in the master dataset.

This dataset will be merged with the Delta table using Delta Lake's MERGE operation

```
05:06 PM (1s) 20
from pyspark.sql.functions import col

master_df = spark.read.format("delta").load(delta_path)

update_products = master_df.limit(5)

update_products = update_products \
    .withColumn("rating", col("rating") + 0.2)
```

> master_df: pyspark.sql.connect.dataframe.DataFrame = [product_id: integer, title: string ... 22 more fields]
> update_products: pyspark.sql.connect.dataframe.DataFrame = [product_id: integer, title: string ... 22 more fields]

05:17 PM (1s) 21

```
from pyspark.sql.functions import lit

new_product_1 = (
    master_df.limit(1)
    .withColumn("product_id", lit(5000001))
    .withColumn("title", lit("Gaming Keyboard"))
    .withColumn("rating", lit(4.8))
)

new_product_2 = (
    master_df.limit(1)
    .withColumn("product_id", lit(5000002))
    .withColumn("title", lit("Wireless Mouse"))
    .withColumn("rating", lit(4.6))
)

new_product_3 = (
    master_df.limit(1)
    .withColumn("product_id", lit(5000003))
    .withColumn("title", lit("USB-C Hub"))
    .withColumn("rating", lit(4.5))
)
```

05:17 PM (2s) 22

```
incremental_df = (
    update_products
    .unionByName(new_product_1)
    .unionByName(new_product_2)
    .unionByName(new_product_3)
)

display(incremental_df)
```

> [See performance \(1\)](#) ✓ Checked for improvements
> incremental_df: pyspark.sql.connect.dataframe.DataFrame = [product_id: integer, title: string ... 22 more fields]

	product_id	title	product_description	rating
1	17198778	H&M	Beige & Tan Solid Stoneware Soap Dispenser	4.9
2	19219450	Susie	Black Solid Cotton T-Shirt Bra - Anti-Odour	4.7
3	18223842	max	Plus Size Non Padded Bra	4.5
4	18100576	MINUTIAE	Women Gold-Toned & White Brass Crystals Handcrafted Gold-Plated Bangle-Style Bracelet	3.6000000000000000...
5	21860658	EROTISSCH	Women Printed Laced Basic Briefs	0.4
6	5000001	Gaming Keyboard	Beige & Tan Solid Stoneware Soap Dispenser	4.8
7	5000002	Wireless Mouse	Beige & Tan Solid Stoneware Soap Dispenser	4.6

```
05:17 PM (3s) 23
incremental_path = "/Volumes/workspace/default/shopping_dataset/customer_incremental"

incremental_df.write \
    .mode("overwrite") \
    .option("header", True) \
    .csv(incremental_path)
```

[See performance \(1\)](#) ✓ Checked for improvements

5. Applied the Delta Lake MERGE operation.

The MERGE operation performs an UPSERT:

- Existing records are updated.
- New records are inserted.

```
05:17 PM (6s) 25 Python
from delta.tables import DeltaTable

delta_table = DeltaTable.forPath(spark, delta_path)

(delta_table.alias("target")
 .merge(
     incremental_df.alias("source"),
     "target.product_id = source.product_id"
 )
 .whenMatchedUpdateAll()
 .whenNotMatchedInsertAll()
 .execute())
```

[See performance \(1\)](#) ✓ Checked for improvements

DataFrame[num_affected_rows: bigint, num_updated_rows: bigint, num_deleted_rows: bigint, num_inserted_rows: bigint]

6. Validated row counts and duplicate Product IDs.

Validation

The updated Delta table is validated by checking:

- Total number of rows
- Duplicate product IDs

```
05:17 PM (1s) 27
final_df = spark.read.format("delta").load(delta_path)

print("Final Row Count:", final_df.count())
```

[See performance \(1\)](#) ✓ Checked for improvements

final_df: pyspark.sql.connect.dataframe.DataFrame = [product_id: integer, title: string ... 22 more fields]

Final Row Count: 1006

```
05:17 PM (1s) 28
duplicates = final_df.groupBy("product_id") \
    .count() \
    .filter("count > 1")

display(duplicates)
```

[See performance \(1\)](#) ✓ Checked for improvements

duplicates: pyspark.sql.connect.dataframe.DataFrame = [product_id: integer, count: long]

product_id	count
------------	-------

No rows returned

0 rows | 0.810s runtime Refreshed 1 hour ago

7. Displayed the final Delta table.

	product_id	title	product_description	rating	ratings_count	initial
5	999003	USB-C Hub	Beige & Tan Solid Stoneware Soap Dispenser	4.5	42	
6	1341220	Miss Chase	Black & White Striped Belted Dress	4.4	22	
7	1376949	F Gear	Unisex Navy & Grey Printed Burner Backpack	4.4	1052	
8	1605539	Karwin	Beige Thermal Semi-Long Leggings	4	76	
9	1708108	GIORDANO	Women Rose Gold-Toned Mirrored Analogue Watch FA2052-55	2.8	5	
10	1728397	Mactree	Men Brown Solid Mid-Top Flat Boots	3.9	1727	
11	1848495	Teakwood Leathers	Men Brown Leather Formal Shoes	3.5	25	
12	1993251	FCUK	Yellow Attitude Trunks CTR10	3.9	26	
13	2184914	SUGAR	Smudge Me Not Liquid Matte Lipstick with Vitamin E 4.5 ml - Greige Rage 11	4.3	76	
14	2466414	Daniel Klein	Women Brown Analogue Watch DK11467-6	4.4	940	
15	2511559	Disney by Wear Your Mind	Girls Pack of 2 Printed Tops	3.5	14	
16	4380622	Bhama Couture	Women Black Solid Peplum Top	2.9	15	
17	4441374	Mayra	Women White Solid Top	3.5	24	
18	5000001	Gaming Keyboard	Beige & Tan Solid Stoneware Soap Dispenser	4.8	42	
19						

Results

- Successfully loaded the dataset into Delta Lake.
- Cleaned the dataset.
- Simulated incremental processing.
- Performed MERGE successfully.
- Verified data integrity and absence of duplicate Product IDs.

Conclusion

This project demonstrates how Delta Lake enables reliable incremental data processing using ACID transactions and the MERGE operation. The workflow provides an efficient approach for handling updates and inserts while maintaining data consistency.